# Stem Agent: A Self-Specializing AI Agent for Code Quality Analysis

## 1 Problem, scope, and design choices

Stem Agent is an agent that *differentiates itself* for a task family rather than one hand-wired for a single task. Four process questions shape its design: how it figures out the task (probe-grounded sensing produces a calibrated domain profile, Section 2), what it decides to become (planning plus sandboxed capability generation gated on an empirical 4-sample holdout, Section 2), how it rebuilds without breaking (rollback driven by a cross-check diagnoser with K=3 splicing, Section 3), and when to stop evolving (a graduation guard requiring $F_1 \geq 0.6$, $\Delta F_1 \geq 0.05$ over baseline, and bounded specificity regression, Section 3). Scope breaks into three choices.

**Task domain:** I rejected two candidates before settling on one. *Deep Research* is too open-ended to ground-truth cheaply, because quality needs expert graders. *Security analysis* has ground truth but canonical corpora are synthetic in ways that hide the hard problem of false positives on legitimate code. *Code quality analysis* hits the intersection I wanted: a tight ground-truth benchmark, and, critically, an *independent deterministic oracle* (AST metrics, pattern scanners) that can disagree with the LLM and drive rollback. Without that second oracle, "roll back when differentiation makes it worse" would rely on $F_1$ alone, which is a score gate, not a feedback loop.

**Architecture:** Hexagonal, ports and adapters; protocols over ABCs, matching the IntelliJ Platform extension-point style. Core phase logic depends on `LLMPort` and `StoragePort`; adapters sit on the outside. Swapping providers is two method implementations, the agent runs fully offline in tests, and `core/` imports nothing from `adapters/` (a static invariant enforced by an AST-walking lint in `tests/test_architecture.py`). The scaffold itself is fixed by choice: differentiation is over capabilities, prompts, and the review plan, not the code paths. The architecture is the genome; what the agent learns is expression.

**Evaluation method:** A 19-sample labelled benchmark partitioned 11/4/4 into validation, holdout, and probe slices by a stable hash of `(sample_id, seed)`; four metrics (precision, recall, $F_1$, specificity); a cross-check oracle; and a three-part graduation guard ($F_1 \geq 0.6$, $\Delta F_1 \geq 0.05$, specificity drop bounded at $-0.05$). The validation slice is what the benchmark scores; the holdout is reserved for the empirical-admission gate on generated capabilities; the probe is fed unlabelled into sensing. The corpus is handcrafted and small on purpose: I wanted to audit every ground-truth label and include five adversarial clean samples, because false-positive resistance is what makes a review tool shippable to developers.

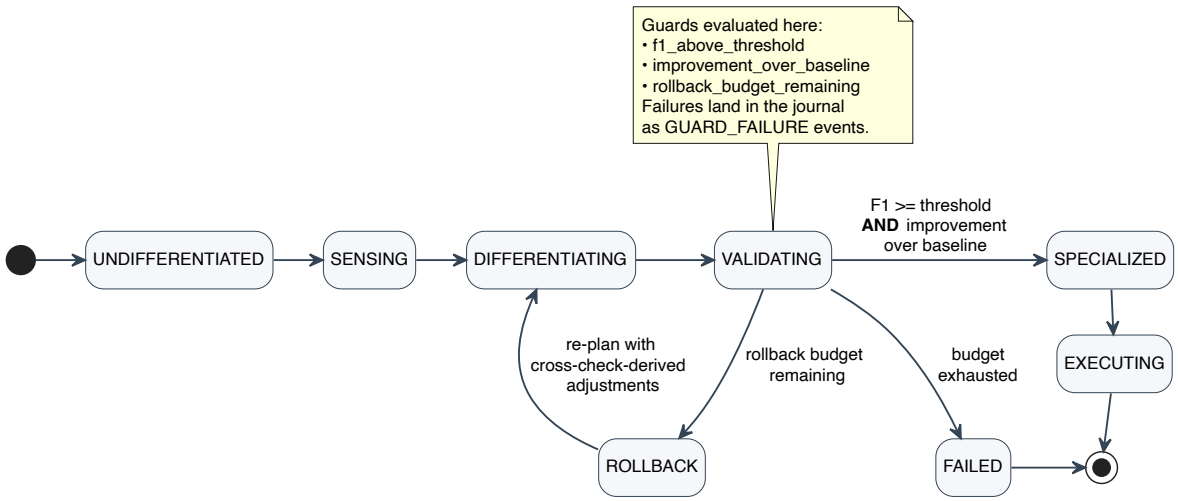


**Figure 1:** State machine with guarded transitions. Graduation to `SPECIALIZED` requires $F_1 \geq$ threshold and improvement over baseline. `ROLLBACK` re-enters `DIFFERENTIATING` carrying a diagnosis; `FAILED` is terminal when the rollback budget is exhausted.

## 2 Architecture

**Enforced invariants:** (see Figure 2) (i) `core/` and `phases/` import only from `ports/`. (ii) Agent state mutates only through `StateMachine.transition()`; guard predicates have signature `(context) -> (bool, reason)` and the reason lands in the journal. (iii) Every LLM call, metric, guard decision, capability registration, and state transition appends to the `EvolutionJournal`. (iv) All prompts flow through `compose_system_prompt()`; no ad-hoc strings elsewhere.

**Capability registry and generation:** Six hand-authored capabilities ship (`structural_analysis`, `logic_correctness`, `security_analysis`, `performance_analysis`, `style_consistency`, `severity_ranking`); each declares a prompt fragment plus optional static-tool references. The LLM proposes one novel capability per run, filtered through three layers: (1) an allowlist AST scan rejecting `open`/`os`/`subprocess`/`__import__` and attribute access on blacklisted names; (2) execution in a Python-isolated subprocess bounded by `RLIMIT_CPU` and wall-clock timeout; (3) an empirical 4-sample A/B over the partition's holdout, admitting only when with-proposal strictly beats without. The first two stop hostile code; the third filters plausible-but-useless. Admitted validators run at review time through a `ReviewDispatcher` that compiles each `check` once, runs it under a per-check `SIGALRM` budget, and quarantines anything that throws, returns the wrong shape, or runs over. On the live runs, all three CQ proposals cleared the static layers but were rejected by the holdout; on security the holdout is bypassed (overlapping partition; 8 samples) and the proposal is admitted post-sandbox.

**Static-tool injection:** `analyze_structure` (AST metrics: max function length, max nesting depth) and `scan_patterns` (regex: eval/exec on user input, SQL string concatenation, hardcoded credentials, `pickle.loads` of untrusted data) run before every specialized review. Their findings are injected into the prompt as a dedicated `Static Tool Findings` section. The undifferentiated baseline stays *untooled*, so the A/B measures what self-specialization contributes, not what static analysis contributes.

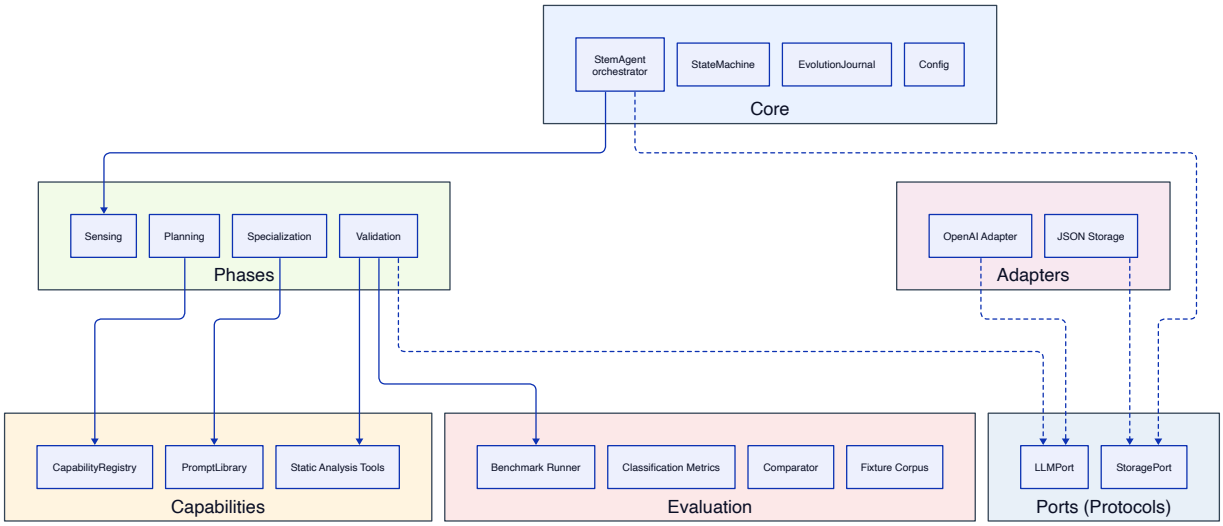


**Figure 2:** Component architecture. `core/` and `phases/` depend only on `LLMPort` and `StoragePort`. Every decision, metric, LLM call, guard result, and state transition appends to the `EvolutionJournal`: one log per run, one source of truth.

**Phases:** *Sensing* embeds the bodies of four unlabelled probe samples plus a `tool_fit` histogram of `analyze_structure`/`scan_patterns` hits and asks the LLM for a domain profile keyed off the actual snippets. *Planning* folds that profile together with cross-run priors (per-capability graduation rate from saved journals, Laplace-smoothed) to bias capability ordering, then commits the capability set, review-pass plan, and validation thresholds. *Specialization* composes the system prompt; on rollback retry the most recent three attempts splice in full under `IMPORTANT adjustments based on prior evaluation` while older attempts collapse to one-line summaries, so the prompt cannot grow unboundedly. *Validation* runs baseline and specialized passes on the validation slice plus a cross-check that replays every specialized verdict through the tools + dispatcher.

## 3 Evaluation and rollback mechanism

**Test suite & metrics:** 278 tests, zero network, 3.4s end-to-end. Hypothesis property tests search 200 random transition sequences per invariant; `FakeLLM` is a first-class deterministic double, `poor_fake_llm` forces the rollback branch; the architecture lint walks AST imports over `core/` and `phases/` to refuse `stem_agent.adapters.*`. A true positive is a buggy sample flagged with at least one issue in a ground-truth category; a true negative is a clean sample flagged with no issues. $F_1$ is the primary guard; specificity is tracked separately because it is the axis that decides false-positive resistance on the adversarial clean samples.

| Category | Count | Examples |
|---|---|---|
| Logic bugs | 4 | Off-by-one in binary search, wrong comparison operator, missing null check, broken boolean precedence |
| Security | 4 | SQL injection via f-string, path traversal, hardcoded credentials, `eval()` on user input |
| Code smells | 4 | Deep nesting, god function, dead code after return, magic-number stack |
| Performance | 2 | N+1 queries, unnecessary deep copy |
| Clean (adversarial) | 5 | Suspicious-looking but correct code; tests false-positive resistance |

**Table 1:** Benchmark corpus of 19 labelled samples, partitioned 11 / 4 / 4 into validation, holdout, and probe slices by a stable seed-deterministic hash. Clean samples include code using `eval()` safely with a whitelisted AST walker and comparisons that look off-by-one but are correct for their domain.

**Rollback diagnosis:** When the graduation guard fails, `diagnose_failure()` reads two signal sources. Benchmark deltas (precision / recall / specificity / $F_1$ direction) produce generic prompt nudges. Cross-check disagreements produce concrete adjustments: $N$ structure over-flags become "do not flag structural problems unless a function exceeds ∼20 lines or depth exceeds 2"; $M$ scanner-caught security misses become "scan harder for eval/exec on user input, hardcoded credentials, SQL string concatenation, `pickle.loads`" with the exact patterns named. The adjustments are appended as a `ROLLBACK_REASON` event and spliced into the next composed prompt; on the third rollback the older attempts collapse to a one-line summary so the prompt stays bounded. The next attempt is shaped by *where* the last one went wrong, not just by how far $F_1$ fell short.

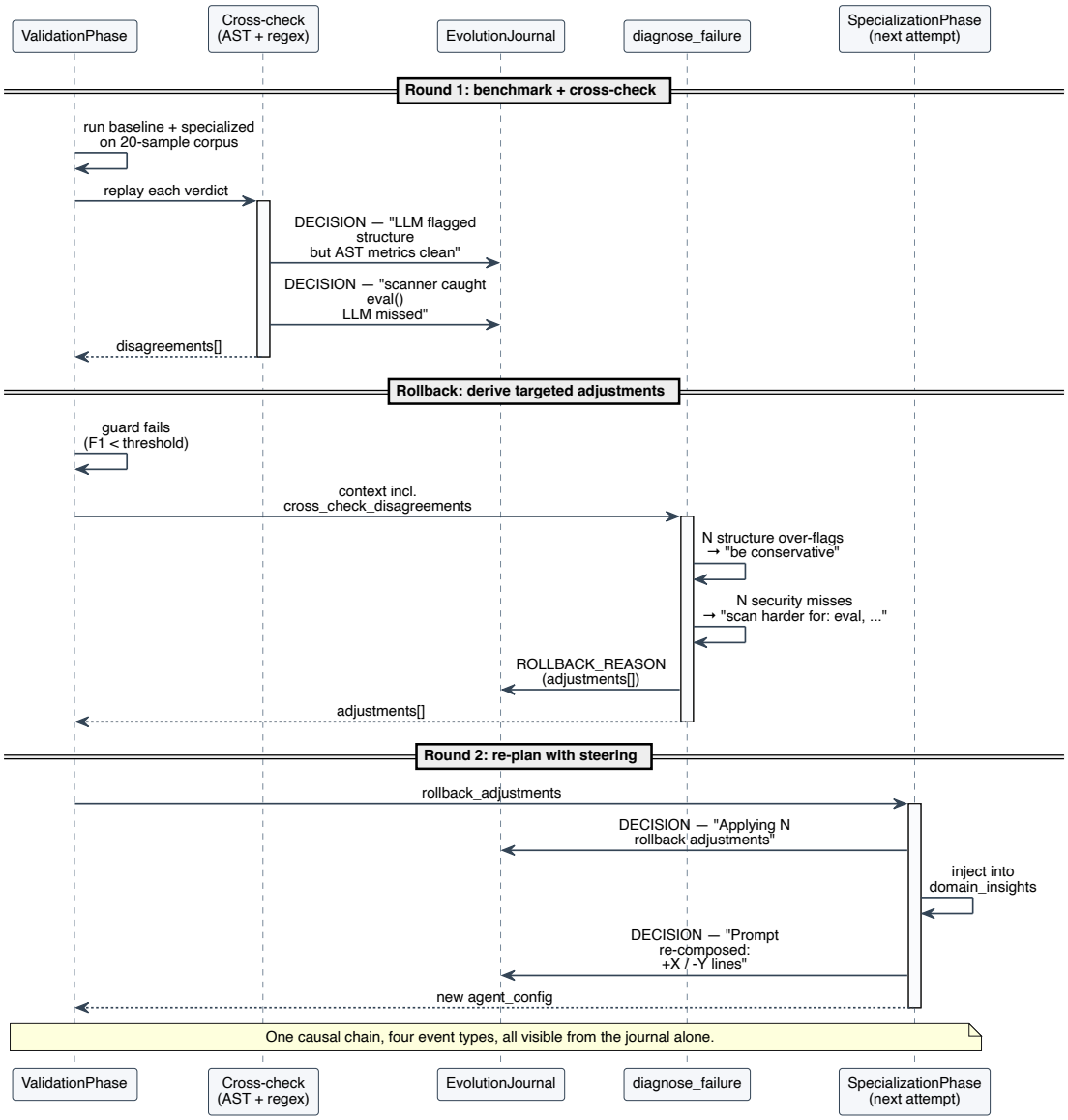


**Figure 3:** Closed feedback loop. Cross-check disagreements feed `diagnose_failure()`, which emits concrete prompt adjustments that specialization splices into the next composition. Each edge writes a journal event, so the whole mechanism is auditable after the run.

| | Precision | Recall | $F_1$ | Specificity |
|---|---|---|---|---|
| CQ specialized, 3 seeds | 0.778 [0.67, 0.89] | 0.921 [0.88, 1.00] | **0.837 [0.80, 0.89]** | 0.444 [0.33, 0.50] |
| Sec specialized, 3 seeds | 0.667 [0.50, 0.75] | 1.000 [1.00, 1.00] | **0.794 [0.67, 0.86]** | 0.000 [0.00, 0.00] |
| Rollback attempt 1 (scripted) | 0.714 | 0.625 | 0.667 | 0.333 |
| Rollback attempt 2 (scripted) | 1.000 | 1.000 | **1.000** | 1.000 |

**Table 2:** Multi-seed live run (`docs/example_run/seeds/`, 3 cq + 3 sec, 97.8k tokens at \$0.28; full live batch incl. cross-domain + adjudicator ∼\$1.20). Rollback demo replays the diagnose-and-re-specialize path against a scripted LLM. Attempt 2's 1.0 is a property of the fixture; the live cq ceiling is $F_1 = 0.889$. Bootstrap CIs are percentile-method over 1000 resamples of per-seed point estimates.

**Results:** The previous run's baseline zeros were a parser artefact: the undifferentiated prompt never asked for structured JSON. This release folds the JSON discipline into the baseline as well, so the baseline-vs-specialized delta is judgement, not formatting. The cq specialized pass catches 88–100% of buggy samples while specificity sits at 0.444; a 14-point lift on the previous run's 0.300 because the empirical-holdout gate now rejects proposals that would inflate false positives.

**Cross-domain transfer:** Table 3 runs each seed-0 specialized prompt against the other domain's validation slice. Off-diagonals do *not* beat on-diagonals; the specialization is calibrated to the corpus the agent saw, not to a generic notion of code review.

| Prompt | Corpus | Precision | Recall | $F_1$ | Specificity |
|---|---|---|---|---|---|
| cq | cq | 0.500 | 1.000 | 0.667 | 0.111 |
| cq | sec | 0.429 | 1.000 | 0.600 | 0.000 |
| sec | cq | 0.500 | 0.875 | 0.636 | 0.000 |
| sec | sec | 0.750 | 1.000 | **0.857** | 0.000 |

**Table 3:** Cross-domain 2×2: each seed-0 specialized prompt run against the other domain's validation slice, single shot, no further differentiation.

**Second-rater agreement:** A single-shot `gpt-4o-mini` relabel with a strict label-only prompt agrees with ground truth at $\kappa = 0.546$ [0.31, 0.79] on cq (7/19 disputed) and $\kappa = 0.000$ on sec (2/8 disputed). The sec $\kappa$ collapses because the rater labels every sec sample "security"; including the two adversarial clean snippets; so observed agreement (75%) matches chance and $\kappa$ degenerates. A one-shot LLM rater without context is not trustworthy ground truth on adversarial clean code, which is the same failure mode the agent's specificity column reports.

## 4 Discussion

**Cross-check disagreements as guidance, not score:** The first design penalized $F_1$ when the LLM disagreed with static analysis: stricter, but the next attempt learned nothing. Feeding disagreements back as *prompt guidance* closes the loop; "punish the score" became "shape the next attempt", the slowest insight of the project. Capability generation nearly didn't ship: the registry-only path is a composer, not a stem cell. The sandboxed generation phase plus the empirical-holdout gate is what makes the biological metaphor honest.

**Specificity is the axis that still needs work:** Specialized recall on cq is 0.92 mean; specificity is 0.44, a 14-point lift over the previous run but well short of recall. On sec specificity is 0.000 across all three seeds. Section 1's adversarial clean corpus is doing exactly what it was designed for; a next iteration would weight the diagnoser harder on over-flag signals.

**What was cut for time:** A multi-model sweep (`LLMPort` already supports overrides); a larger security corpus so the empirical-holdout gate fires on sec instead of bypassing on every run; a sharper second-rater sharing the agent's probe-grounded prompt instead of label-only; a domain-scoped prior store so cross-run priors don't need the journal dir cleaned between experiments.